

README

Helix Proxy — Security & Zero-Trust Model (DESIGN)

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z **Status:** DESIGN / config-plan — NOT yet live. Live enforcement (407 auth challenge, tunnel-drop no-leak) is PROVEN in P10, not here (§11.4.6). **Authority:** docs/superpowers/specs/2026-06-30-vpn-aware-proxy-extension-design.md §12 (Security & secrets) + §13 (anti-bluff test strategy). Composes the §11.4 anti-bluff covenant — every claim below is a *plan*, falsifiable by the P10 captured-evidence probes named at the end.

This directory documents the security posture of the VPN-aware dynamic-routing extension. It owns DESIGN only — no live wiring, no credentials, no operator resources touched (§11.4.174).

1. Secrets: the Podman-secrets reference model (§11.4.10)

Rule: secret NAMES live in git; secret VALUES never do.

- `.env.example` declares only `*_SECRET` variables whose VALUES are Podman **secret names** (e.g. `PG_PASSWORD_SECRET=helixproxy_pg_password`, `CONTROL_API_TLS_CERT_SECRET=helixproxy_api_cert`, `CONTROL_API_TLS_KEY_SECRET=helixproxy_api_key`) — never a password, key, or token.
- The real material lives in the `gitignored secrets/` directory and the `gitignored config/htpasswd` (both excluded by `.gitignore`). The operator drops the files there out-of-band.
- `scripts/load_podman_secrets.sh` (the companion loader — see §2) reads those `gitignored` sources and creates the named **rootless** Podman secrets (§11.4.161). Containers reference secrets by name; the value is mounted into the container's `tmpfs` at runtime and is never serialised into an image layer, a compose file, or git.

- Source files SHOULD be `chmod 600` under a `chmod 700` parent (§11.4.10); the loader warns (does not fail) on looser perms and never prints a value.
- On suspected leak: rotate the source, re-run the loader with `--replace`, and follow §11.4.10 rotation.

Secret name (env)	Default name	Gitignored source	Purpose
PG_PASSWORD_SECRET	helixproxy_pg_password	secrets/ pg_password	Postgres control-plane password
CONTROL_API_TLS_CERT_SECRET	helixproxy_api_cert	secrets/ api_cert.pem	control-API mTLS certificate
CONTROL_API_TLS_KEY_SECRET	helixproxy_api_key	secrets/ api_key.pem	control-API mTLS private key
VPN_WG_KEY_SECRET	helixproxy_vpn_wg_key	secrets/ vpn_wg_key	WireGuard tunnel private key
PROXY_HTTPASSWD_SECRET	helixproxy_proxy_htpasswd	config/ htpasswd	Squid per-user htpasswd (hashes)

2. Companion loader — `scripts/load_podman_secrets.sh`

Idempotent, rootless, operator-run. Maps each secret name above to its gitignored source path and creates the secret if absent (re-run is safe; `--replace` recreates, `--dry-run` previews). It contains NO secret values — only the name→path mapping. It refuses to run as root (§11.4.161) and touches ONLY `helixproxy_*` secrets — never the operator’s containers, the `wg0-mullvad` / `lava-*` interfaces, or unrelated secrets (§11.4.174).

```
# operator workflow (out-of-band, real values never in git):
mkdir -p secrets && chmod 700 secrets
printf '%s' "<pg-password>" > secrets/pg_password && chmod 600
secrets/pg_password
htpasswd -B -c config/htpasswd <user> # bcrypt
hashes only
scripts/load_podman_secrets.sh # create the named
rootless secrets
```

```
scripts/load_podman_secrets.sh --dry-run # idempotent preview on re-run
```

3. Per-user proxy auth + audit log

- **Auth template:** config/squid/templates/auth.conf.tmpl adds HTTP Basic per-user auth as an **additive include** (auth_param basic against the gitignored httpasswd → acl authenticated proxy_auth REQUIRED → http_access deny !authenticated), gated by PROXY_AUTH_ENABLED (§11.4.122 — the base squid.conf is never modified).
 - **Composition:** with the dynamic-routing include also active, the effective policy is “**authenticated user AND tunnel up**” — each include contributes its own fail-closed deny guard above the base allow rules.
 - **Audit log:** the control-plane already owns an audit_log table (Stream A schema). Per-user requests are attributable by the authenticated username (Squid %un access-log token feeding the control-plane), so each routed / denied request maps to a user + a chosen vpn_profile for the audit trail. (Live wiring of the access-log → audit_log pipe is a control-plane stream; out of this config-plan’s scope.)
-

4. Kill-switch: gluetun’s BUILT-IN firewall (§11.4.74 — configure, don’t reimplement)

The egress kill-switch is **gluetun’s own** firewall — we CONFIGURE it, we do not build our own. Set FIREWALL=on (gluetun default) on every tunnel container: gluetun installs nftables/iptables rules inside its own network namespace that permit egress ONLY through the active VPN interface, so if the tunnel drops there is no path for target traffic to leak onto the real uplink. Squid routes through the gluetun container as its cache_peer parent and is fail-closed at the application layer too (never_direct allow all + http_access deny !tun_up → branded 503 in dynamic-routing.conf.tmpl). The two layers are independent: gluetun’s firewall is the network-layer guarantee; Squid’s fail-closed routing is the application-layer guarantee.

5. mTLS on the control-API

The control-API is served with mTLS using the `helixproxy_api_cert / helixproxy_api_key` Podman secrets (cert + key by name, never in git). Clients (admin UI, CLI) present a client cert; the API verifies against the CA. Design only here — the API server lives in `control-plane/` (other streams).

6. P10 no-leak / auth-enforced test DESIGN (anti-bluff — §11.4 / §11.4.69 / §11.4.107 / §11.4.123)

These are the captured-evidence probes that will PROVE the above LIVE in P10. Stated now as design; **none is claimed proven yet** (§11.4.6).

- **Unauthenticated → 407 (auth enforced).** With `PROXY_AUTH_ENABLED=true: curl -x http://proxy:53128 http://example.com` with NO credentials → capture HTTP **407 Proxy Authentication Required** + the `Proxy-Authenticate: Basic realm=...` header; THEN with valid `-U user:pass` → 200 + real body. PASS requires BOTH the negative (407) and positive (200) captures — a 200 alone does not prove the gate exists.
 - **Tunnel-drop → zero egress (kill-switch no-leak).** Bring a tunnel up, then drop it; run `tcpdump` **inside the gluetun network namespace on the real uplink interface** for the target host/IP while issuing requests → capture **zero** target packets on the real uplink + DNS only via the intended resolver, and Squid returns the branded 503. Per §13: *leak-testing while the tunnel is up proves nothing* — the drop is the test.
 - **Self-validated analyzer (§11.4.107(10)):** the leak/auth analyzers ship with a golden-good and a golden-bad fixture; the golden-bad (a captured leak / a missing 407) **MUST FAIL**, or the probe is a bluff gate.
 - **Evidence path:** captured under `qa-results/` (raw, gitignored) → curated to `docs/qa/<run-id>/` at P10 (§11.4.83).
-

7. What is proven NOW vs owed to P10 (§11.4.6)

Claim	Now	P10
Auth directives + routing include	 proven	—

Claim	Now	P10
PARSE & compose (squid -k parse exit 0)		
Loader script parses (sh -n + bash -n) & leaks no value	✓ proven	—
No secret value in any tracked file	✓ proven (pre- store grep)	—
Unauthenticated request → 407, valid → 200	🕒 design	✓ live capture
Tunnel drop → zero egress on real uplink (in-netns tcpdump)	🕒 design	✓ live capture
mTLS handshake on control-API	🕒 design	✓ live capture